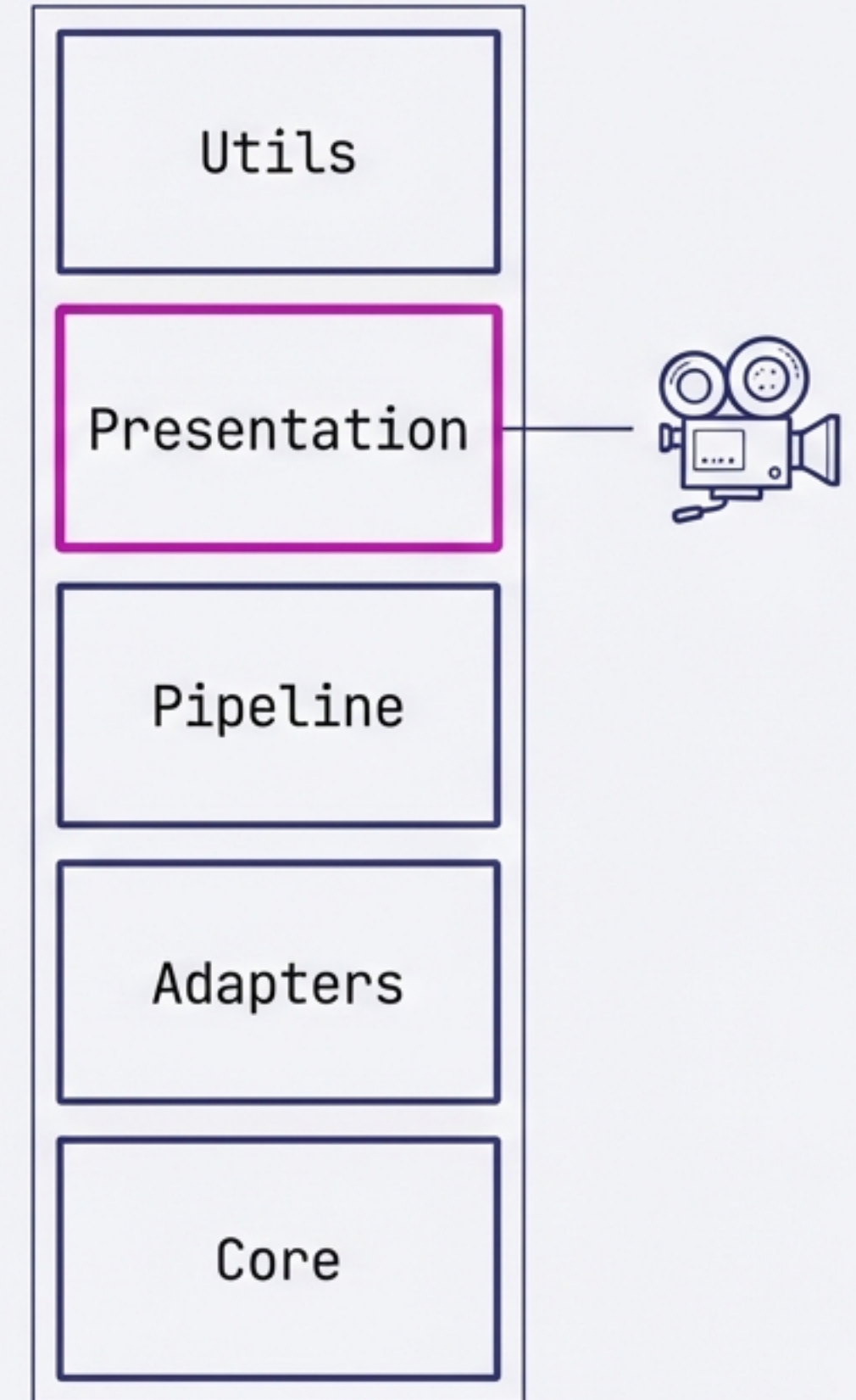


Bakery & Luna: Arquitectura Modular para Visión Computacional

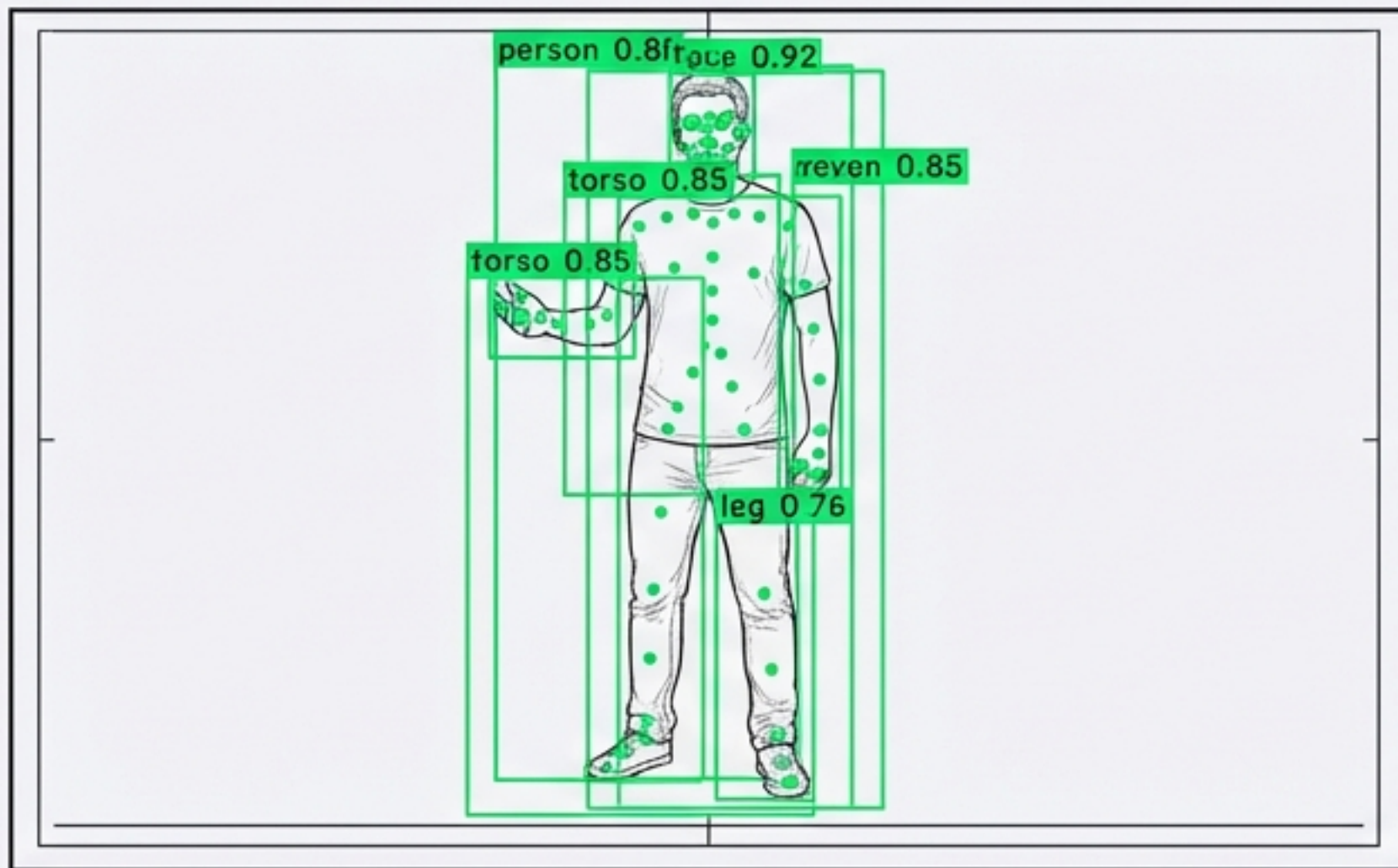
Inferencia dual de modelos YOLO
con principios SOLID y estética
cinematográfica.

Presentación Técnica | Octubre 2023 | Autor: Equipo de Ingeniería Bakery



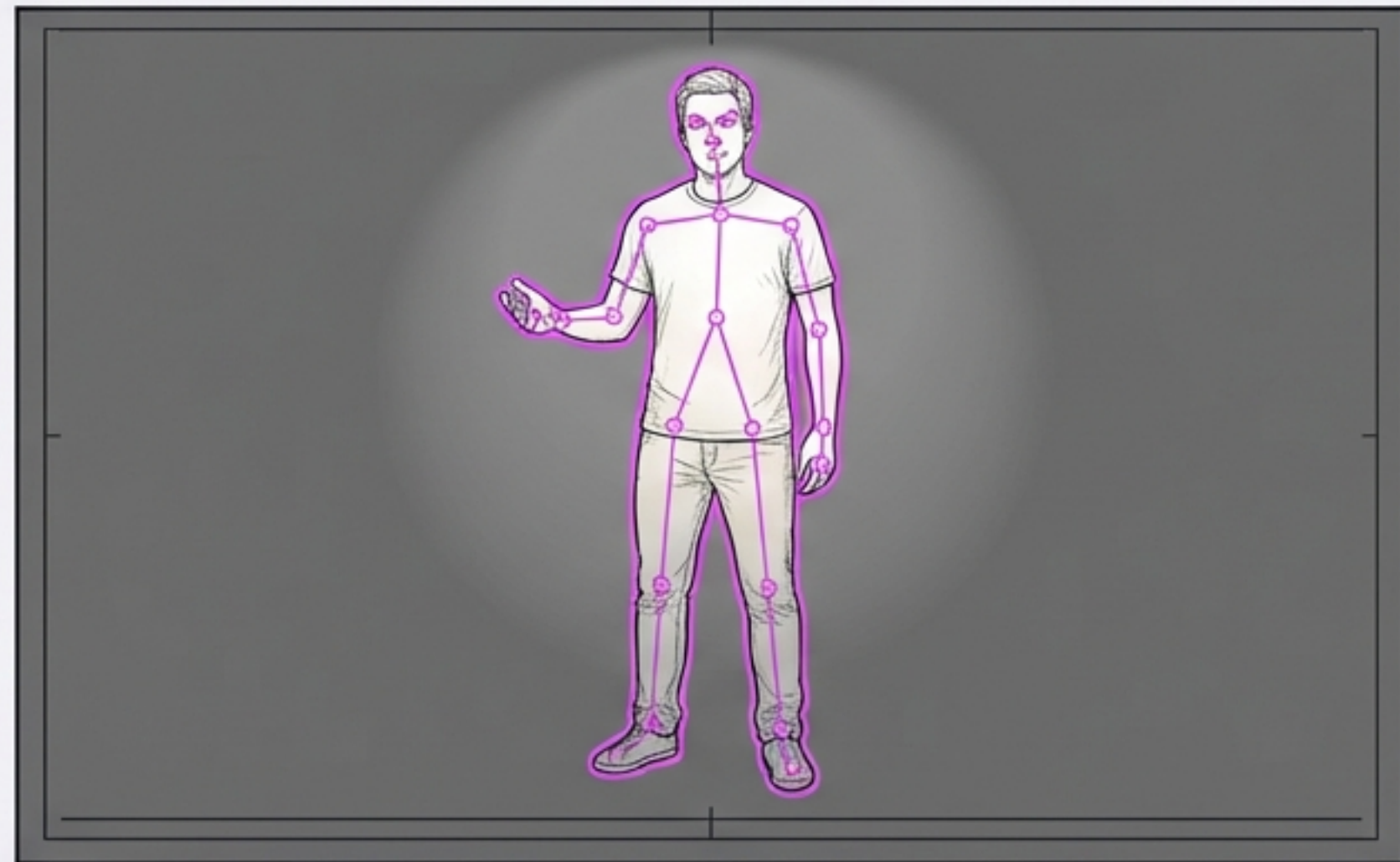
“La Experiencia Luna: Más allá de la Inferencia Cruda”

Raw Output



Salida Estándar YOLO: Funcional pero ruidosa.

Luna Experience



Salida Luna: Estética Roger Rabbit con enfoque selectivo.

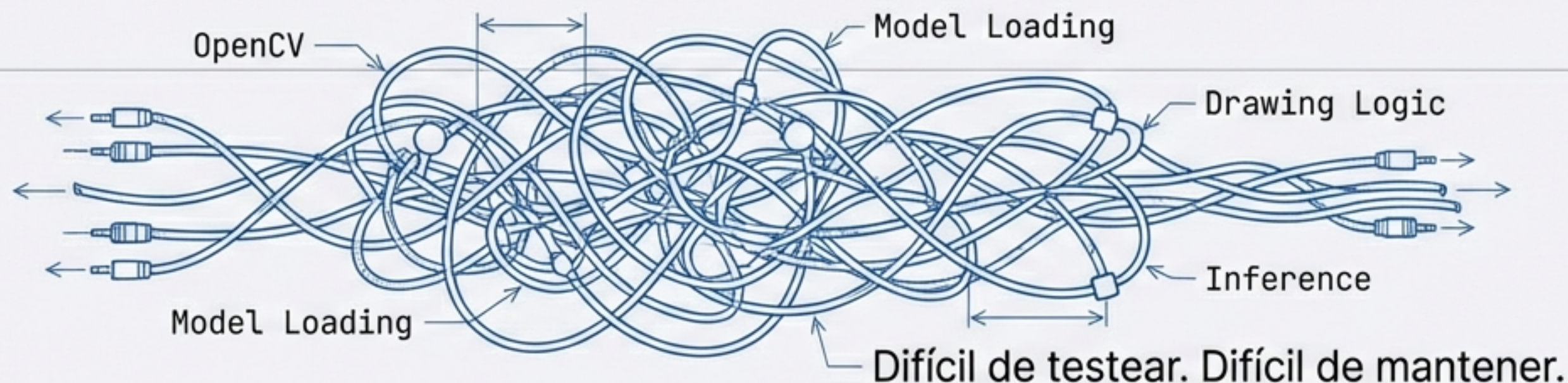
El Concepto: Ejecución simultánea de Segmentación de Instancias y Estimación de Postura (17 keypoints).

La Diferencia: Fondo desaturado, sujetos en color, y “Spotlight” dinámico para guiar la atención.

El Desafío: Complejidad vs. Rendimiento

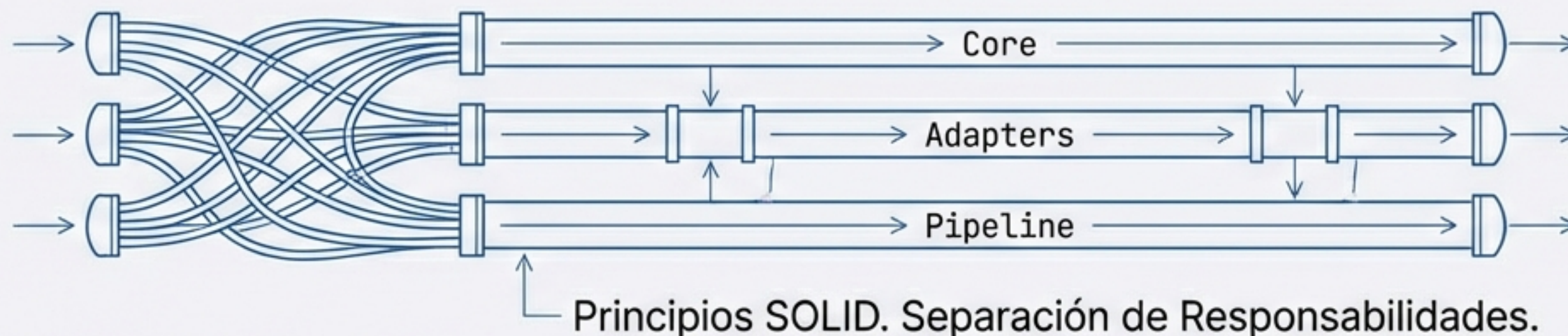
The Problem

El problema del
"Código Espagueti".



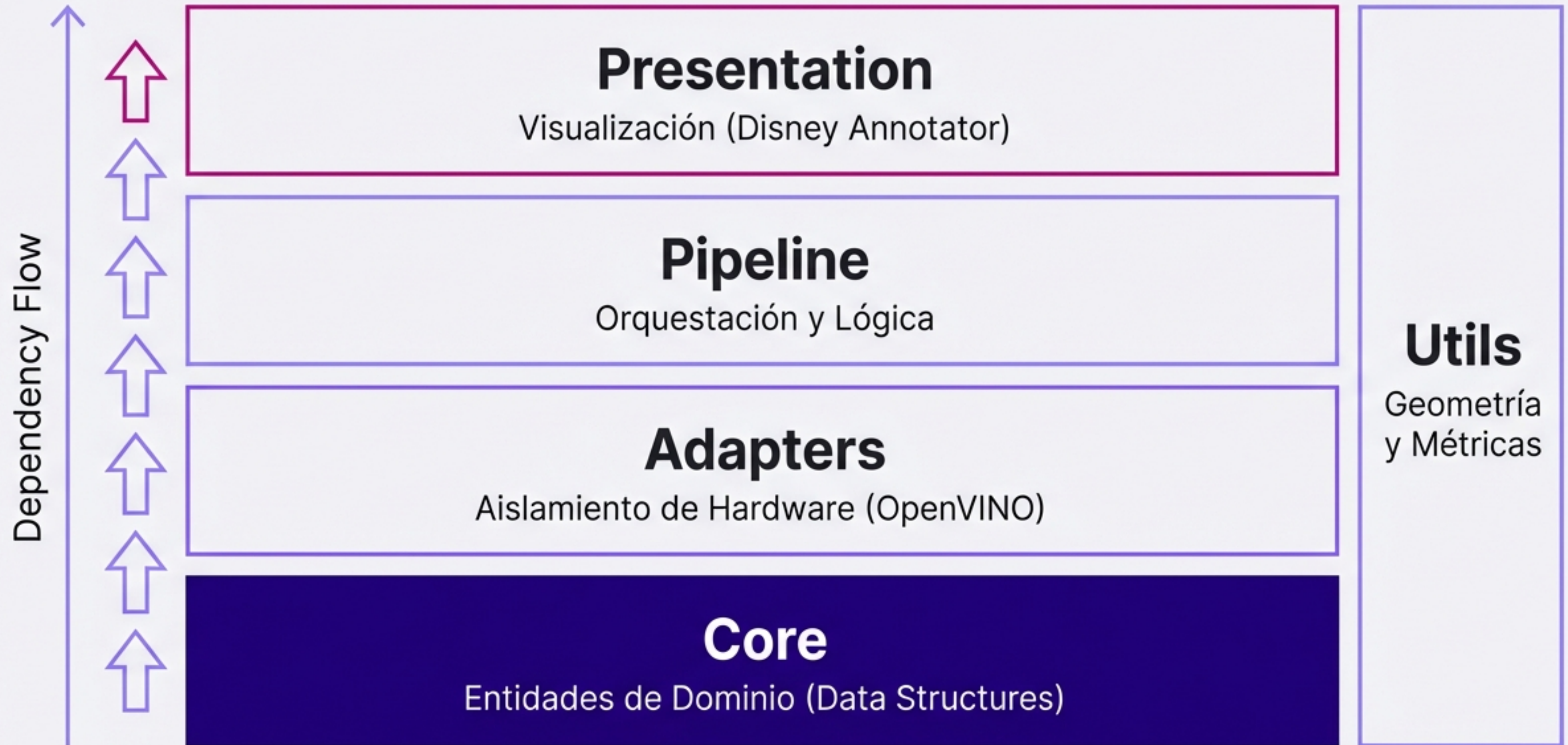
The Solution

La Solución:
Framework Bakery.



- Arquitectura en 5 capas.
- Optimización inteligente (Caching & Scheduling).
- Flujo de datos predecible.

Arquitectura en 5 Capas



Capa Core: La Verdad Inmutable

El corazón del sistema define las estructuras de datos. Sin dependencias externas.

Value Objects: Una vez creado un objeto (como un esqueleto detectado), no se puede modificar.

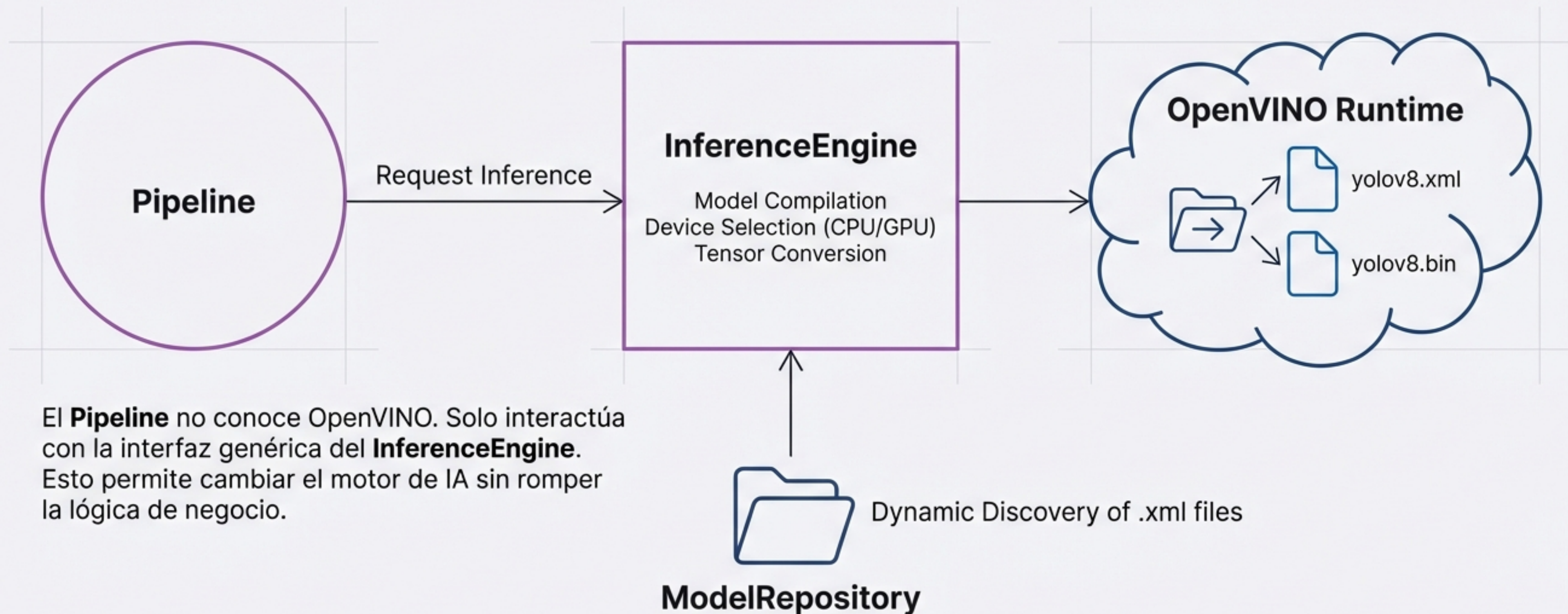
Beneficio:

Previene efectos secundarios y hace el flujo de datos totalmente predecible.

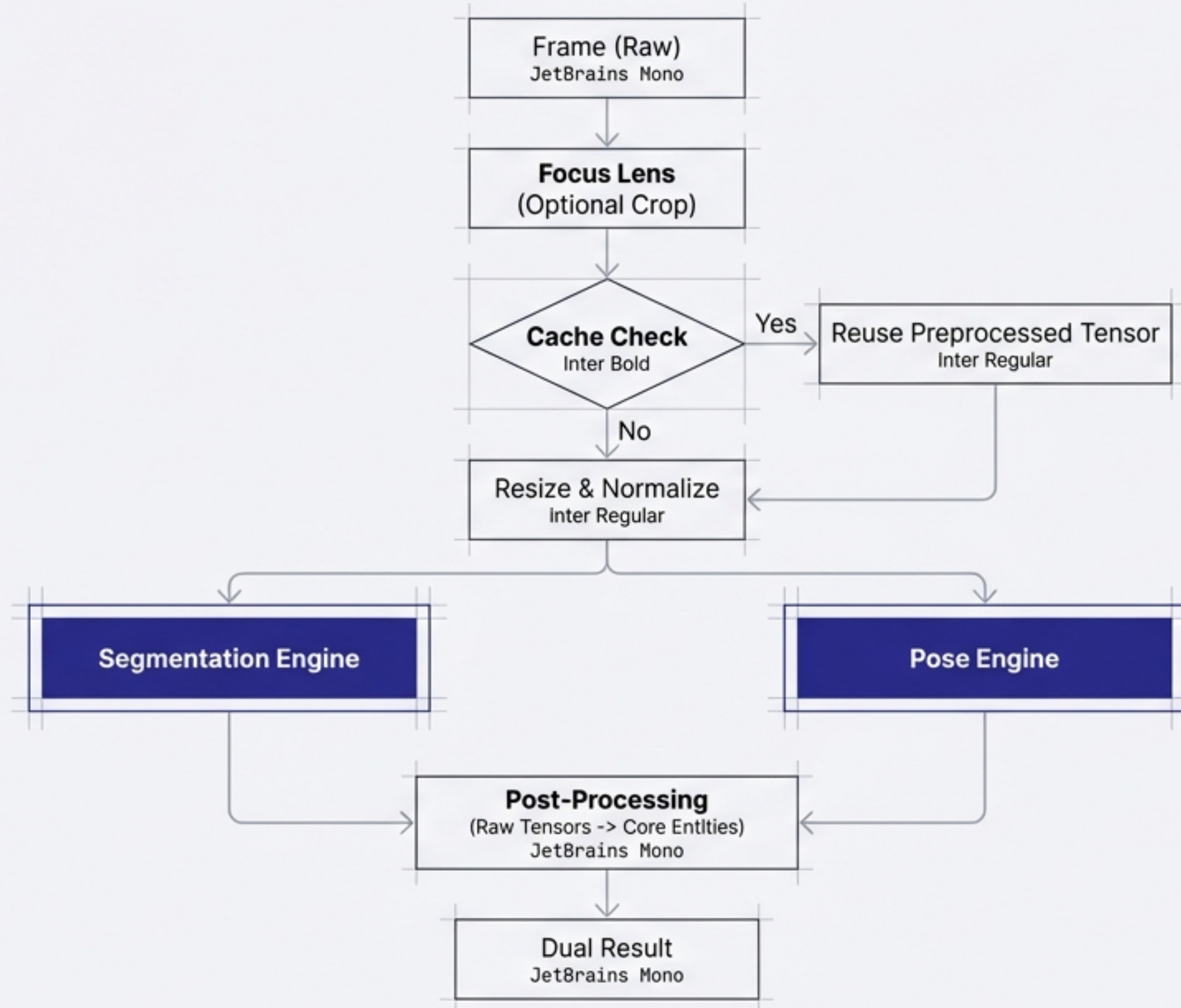
```
@dataclass(frozen=True) ← Inmutabilidad garantizada
class BoundingBox:
    """Representación inmutable de una detección."""
    x1: float
    y1: float
    x2: float
    y2: float
    confidence: float
    class_id: int

    @property
    def xyxy(self) -> np.ndarray:
        return np.array([self.x1, self.y1, self.x2, self.y2])
```


Inter Tight Bold, tracking -20, (#1D1D1F): Capa Adapters: Aislamiento del Hardware



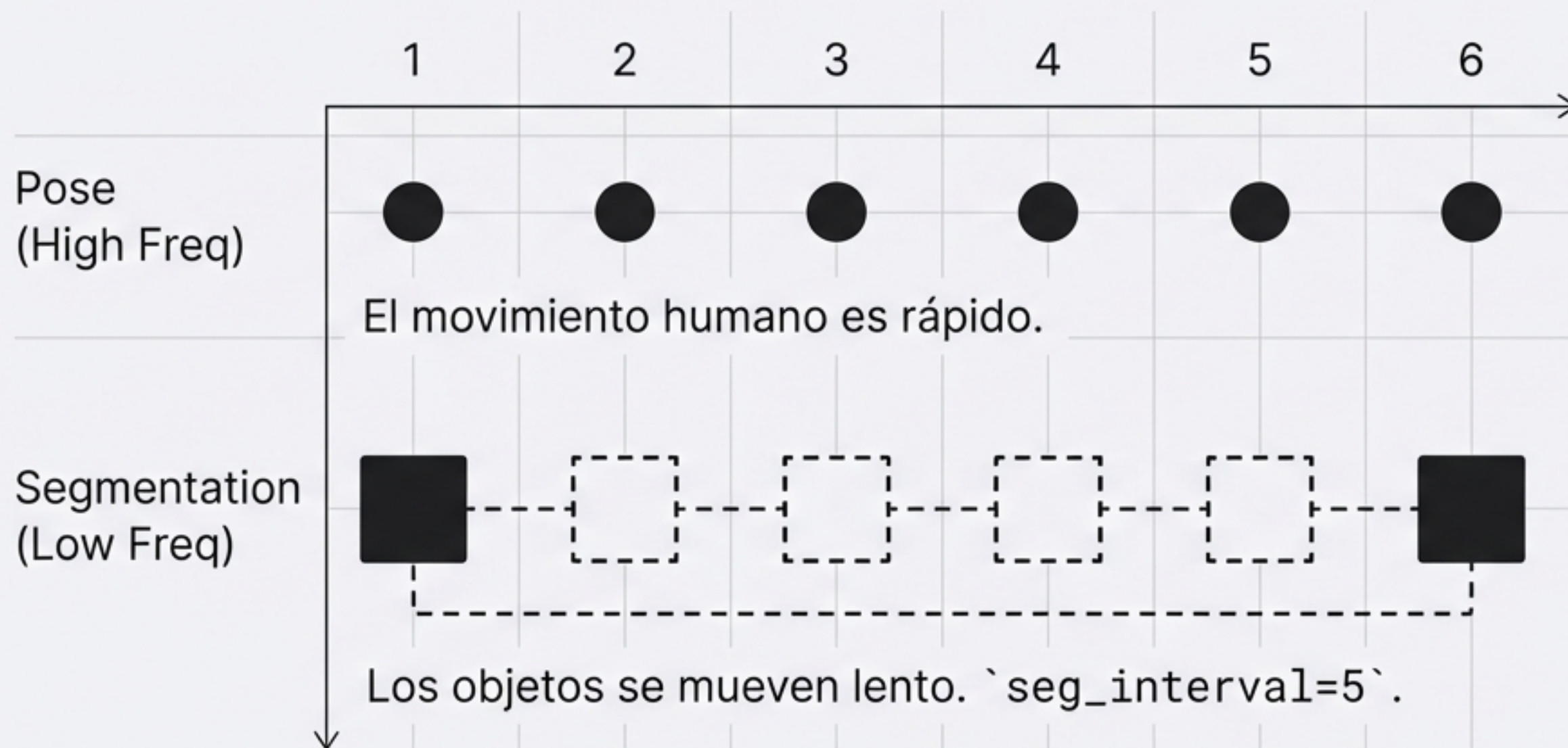
Capa Pipeline: Orquestación Dual



Optimización de Cache:

Si ambos modelos comparten resolución (ej. 640x640), se reutilizan los tensores. Ahorro de cómputo significativo.
Inter Regular

Optimización: Smart Segmentation Scheduling

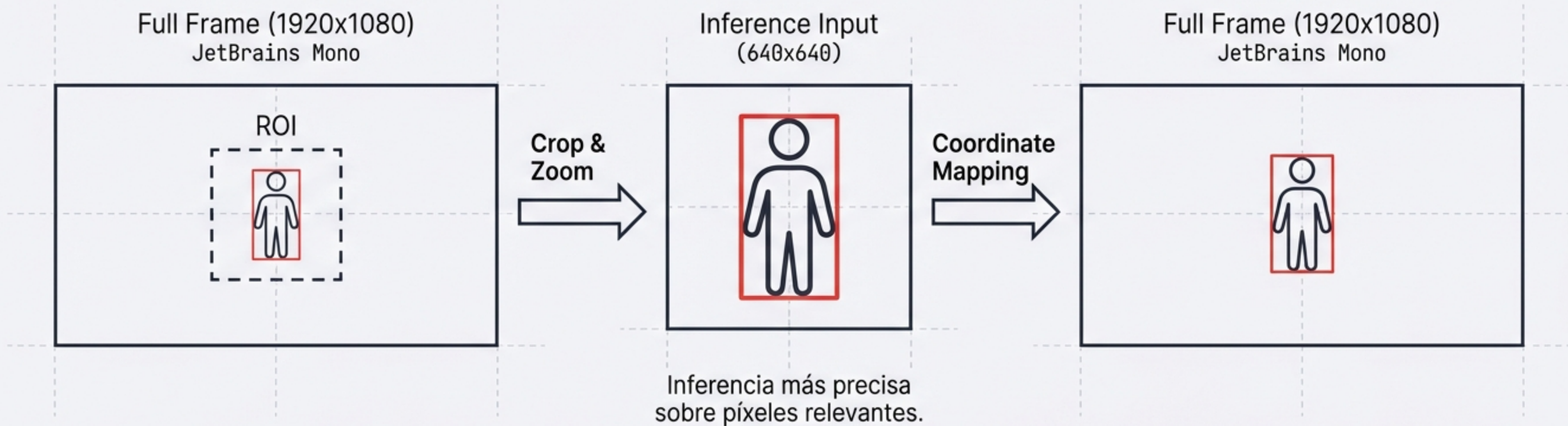


Resultado:

Reducción del 80% en carga de segmentación.

Los resultados se 'cachean' y reutilizan en los frames intermedios.

Optimización: Sistema "Focus Lens"

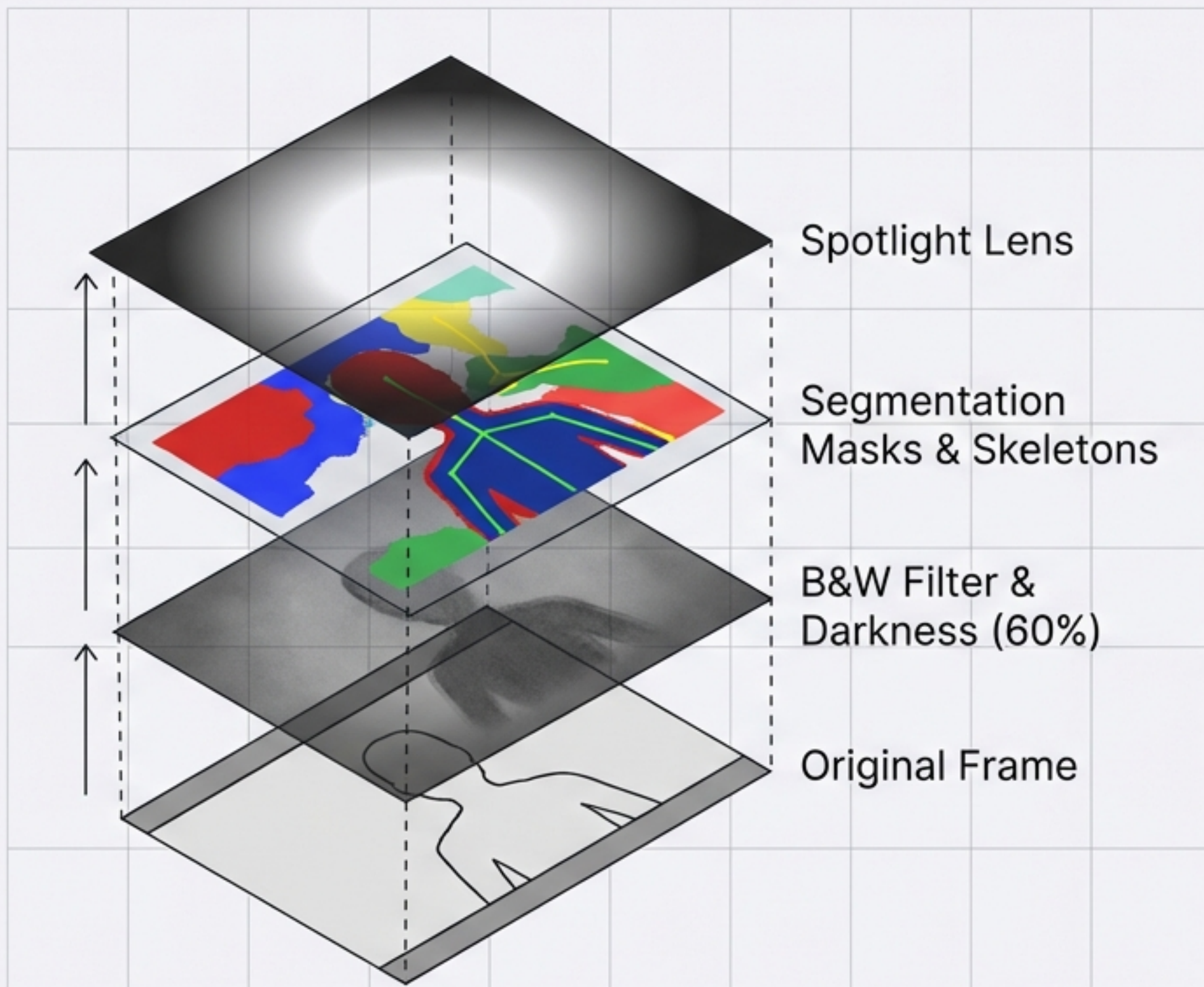


Estrategias:

- * **Zoom:** Escala la región (mantiene aspect ratio).
- * **Pad:** Añade bordes negros (mantiene escala original).

La capa `Utils` maneja la matemática de re-mapeo automáticamente.

Capa Presentation: Renderizado en 8 Capas



- **DisneyAnnotator Pipeline:**

1. Fondo oscurecido para contraste.
2. Superposición de máscaras y esqueletos en color.
3. Efecto de lente (Spotlight) para guiar el ojo.

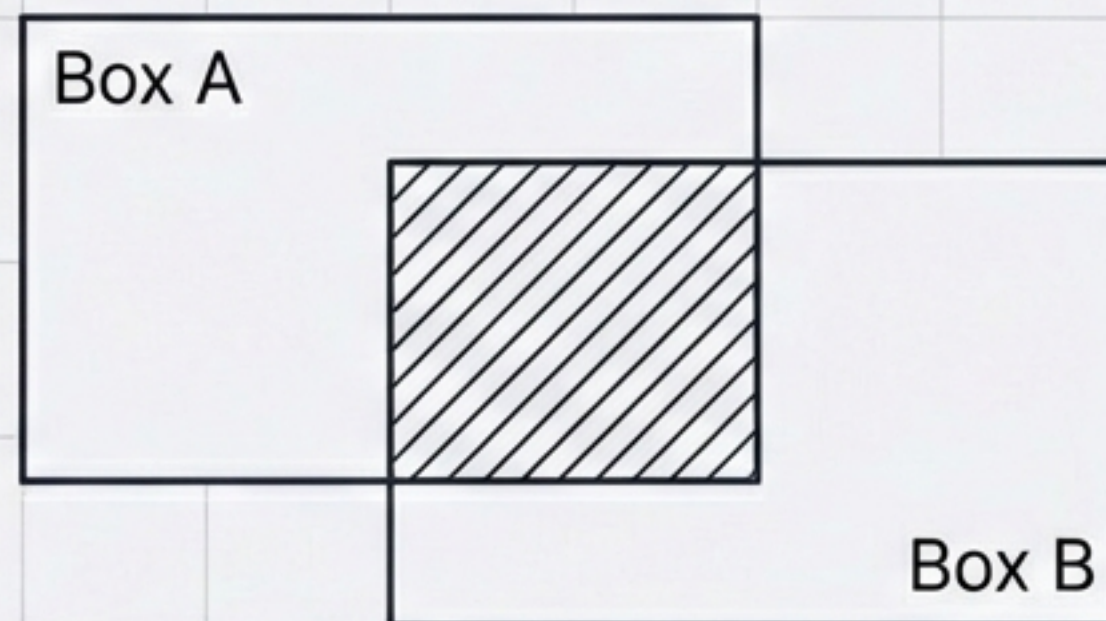
Todo configurable via `RenderConfig`.

Capa Utils: Geometría y Métricas

> Performance Report

Total Frames: 1200
Seg Efficiency: 5.0x (Cached 80%)
Avg FPS: 42.5

Clase `PerformanceMetrics` rastrea la eficiencia real.



$$\text{IoU} = \frac{\text{Area Intersección}}{\text{Area Unión}}$$

Cálculo de IoU para supresión de duplicados (NMS).

Configuración Robusta y "Fail-Fast"

Uso de **Dataclasses congeladas** con **validación inmediata** (`__post_init__`). Si la configuración es inválida, el sistema se detiene antes de procesar el primer frame.

```
@dataclass(frozen=True)
class RenderConfig:
    bw_darkness: float = 0.6
    spotlight_brightness: float = 1.3

    def __post_init__(self):
        if not (0.0 <= self.bw_darkness <= 1.0):
            raise ValueError("De between 0.0 and 1.0")
```



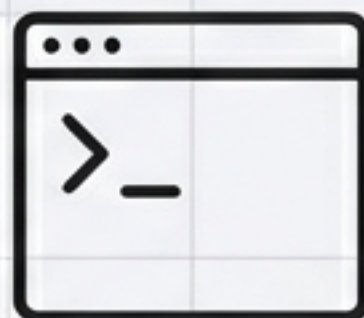
Validación en tiempo de construcción

Experiencia de Desarrollo (DX)



Auto-Discovery

Escaneo de `.xml`
en carpetas.



CLI Unificado

```
python run_luna.py
```



Source-Only

Modelos pesados
ignorados en `.gitignore`.

```
$ python run_luna.py --video sample.mp4 --seg-interval 5  
> Models discovered: YOLOv8-Seg (FP16), YOLOv8-Pose (FP16)  
> Pipeline initialized. Processing...
```


Stack Tecnológico

Python 3.11

JetBrains Mono

Type Hinting agresivo
& Dataclasses.

OpenVINO 2024

JetBrains Mono

Inferencia optimizada
en CPU/GPU.

OpenCV & Supervision

JetBrains Mono

Procesamiento de
imagen y utilidades.

Ultralytics YOLO

JetBrains Mono

Arquitectura base de
los modelos.

Pytest

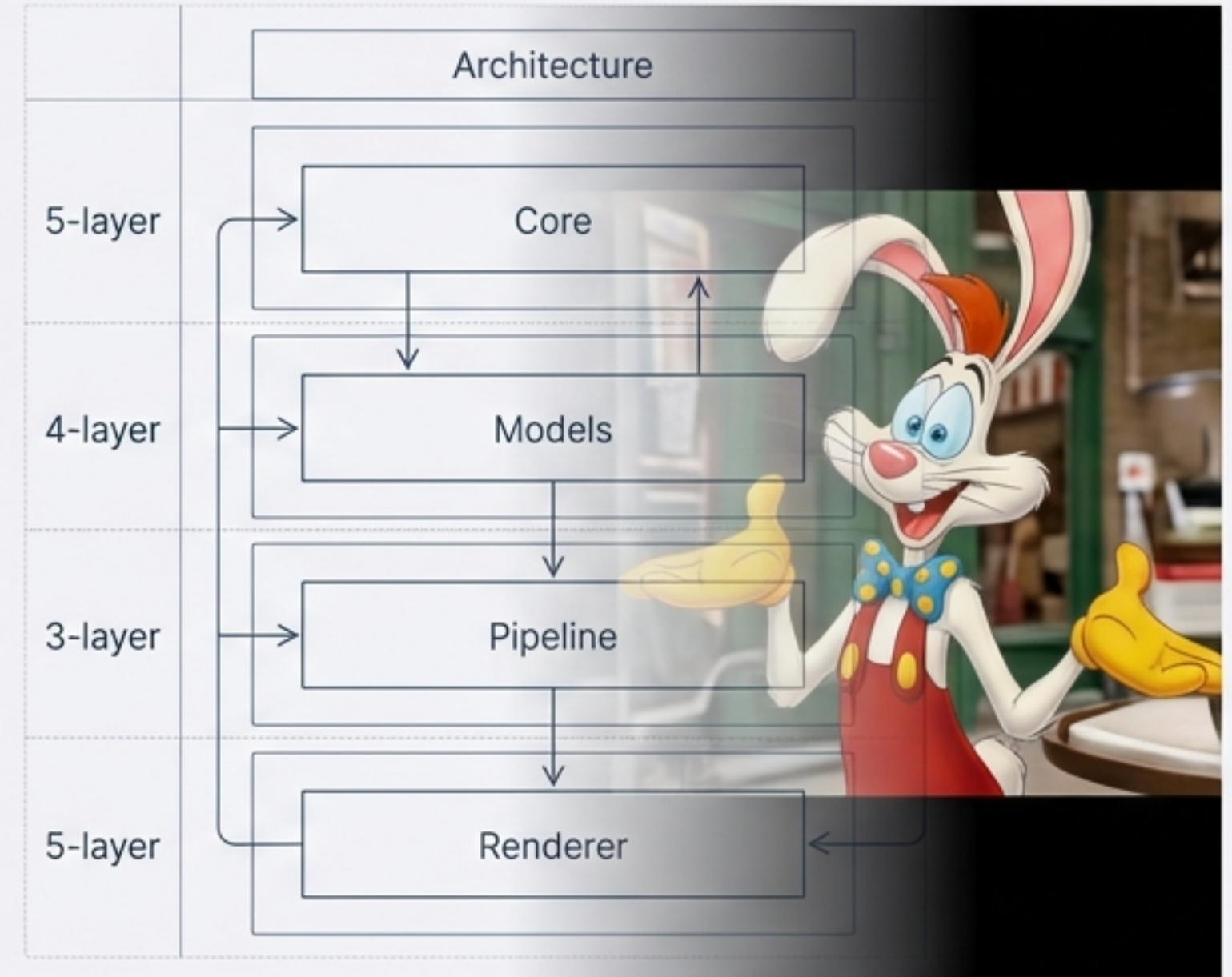
JetBrains Mono

Tests unitarios y de
integración.

Conclusión: El Código como Arte

Bakery demuestra que el software de visión artificial puede ser limpio, mantenible y robusto.

- **Arquitectura:** SOLID y Capas.
- **Rendimiento:** Caching inteligente.
- **Resultado: Luna**, una experiencia visual de alta gama.



Repositorio: ``bakery-luna`` | Documentación completa en ``/docs``